

Dynamic Programming in C/C++: Code Supplement

This document contains all the C/C++ code samples from the "Dynamic Programming: From Recursion to Optimization" lecture. Each sample is a complete, compilable program with necessary headers and a `main` function to demonstrate its usage.

1. Introduction to Recursion & its Pitfalls

1.1. Fibonacci — Naive Recursion

This is the classic, inefficient recursive solution. It demonstrates the concept of a recurrence relation but suffers from exponential time complexity due to re-computing the same values.

C++

```
#include <iostream>

int fib(int n) {
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    std::cout << "fib(10) = " << fib(10) << std::endl;
    return 0;
}
```

1.2. Timing the Naive Approach

This program measures the execution time of the naive Fibonacci function, clearly showing the exponential slowdown as `n` increases.

C++

```
#include <iostream>
#include <vector>
#include <chrono>

long long fib(int n) {
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}
```

```

}

int main() {
    std::vector<int> test_values = {10, 20, 30, 35, 40, 42};
    for (int n : test_values) {
        auto start = std::chrono::high_resolution_clock::now();
        long long result = fib(n);
        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double> elapsed = end - start;
        std::cout << "fib(" << n << ") = " << result
                  << ", Time: " << elapsed.count() << "s" << std::endl;
    }
    return 0;
}

```

1.3. Climbing Stairs — Naive Recursion

This problem is structurally identical to Fibonacci. The number of ways to climb n stairs is the sum of ways to climb $n-1$ and $n-2$ stairs.

C++

```

#include <iostream>

int climbStairs(int n) {
    if (n <= 1)
        return 1;
    return climbStairs(n - 1) + climbStairs(n - 2);
}

int main() {
    std::cout << "Ways to climb 10 stairs: " << climbStairs(10) << std::endl;
    return 0;
}

```

1.4. Grid Paths — Naive Recursion

This demonstrates a 2D recurrence. The number of paths to cell (m, n) is the sum of paths to $(m-1, n)$ and $(m, n-1)$.

C++

```

#include <iostream>

int countPaths(int m, int n) {
    if (m == 1 || n == 1)

```

```

        return 1;
    return countPaths(m - 1, n) + countPaths(m, n - 1);
}

int main() {
    std::cout << "Paths in a 3x3 grid: " << countPaths(3, 3) << std::endl;
    return 0;
}

```

2. Core DP Techniques

2.1. Fibonacci — Memoization (Top-Down)

We store the results of expensive function calls in a cache (an array or map) and return the cached result if the same input occurs again.

C-Style (Array)

C++

```

#include <iostream>
#include <vector>
#include <cstring> // For memset

int memo[100];

int fib_memo(int n) {
    if (memo[n] != -1)
        return memo[n];
    if (n <= 1)
        return n;
    memo[n] = fib_memo(n - 1) + fib_memo(n - 2);
    return memo[n];
}

int main() {
    memset(memo, -1, sizeof(memo));
    std::cout << "fib_memo(50) = " << fib_memo(50) << std::endl;
    return 0;
}

```

C++ Style (unordered_map)

C++

```

#include <iostream>
#include <unordered_map>

std::unordered_map<int, long long> cache;

long long fib_memo_map(int n) {
    if (cache.count(n))
        return cache[n];
    if (n <= 1)
        return n;
    cache[n] = fib_memo_map(n - 1) + fib_memo_map(n - 2);
    return cache[n];
}

int main() {
    std::cout << "fib_memo_map(50) = " << fib_memo_map(50) << std::endl;
    return 0;
}

```

2.2. Fibonacci — Tabulation (Bottom-Up)

We build a table (a `dp` array) from the base case up to the desired value, iteratively.

C++

```

#include <iostream>
#include <vector>

long long fib_tab(int n) {
    if (n <= 1)
        return n;

    std::vector<long long> dp(n + 1);
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];

    return dp[n];
}

int main() {
    std::cout << "fib_tab(50) = " << fib_tab(50) << std::endl;
}

```

```
    return 0;
}
```

2.3. Fibonacci — Space Optimized

For problems where the state `dp[i]` only depends on `dp[i-1]` and `dp[i-2]`, we only need to store the last two values.

C++

```
#include <iostream>

long long fib_optimized(int n) {
    if (n <= 1)
        return n;

    long long prev2 = 0, prev1 = 1;

    for (int i = 2; i <= n; i++) {
        long long curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }

    return prev1;
}

int main() {
    std::cout << "fib_optimized(50) = " << fib_optimized(50) << std::endl;
    return 0;
}
```

3. Classic DP Problems

3.1. Climbing Stairs — DP

This is the tabulation solution for the climbing stairs problem.

C++

```
#include <iostream>
#include <vector>

int climbStairs_dp(int n) {
    if (n <= 1)
```

```

        return 1;
    std::vector<int> dp(n + 1);
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];

    return dp[n];
}

int main() {
    std::cout << "Ways to climb 10 stairs (DP): " << climbStairs_dp(10) << std::endl;
    return 0;
}

```

3.2. Coin Change (Fewest Coins)

Find the minimum number of coins to make a given amount.

C++

```

#include <iostream>
#include <vector>
#include <algorithm> // For std::min

int coinChange(const std::vector<int>& coins, int amount) {
    std::vector<int> dp(amount + 1, amount + 1); // Infinity
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i - coin >= 0) {
                dp[i] = std::min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}

int main() {
    std::vector<int> coins = {1, 2, 5};
    int amount = 11;
    std::cout << "Fewest coins for " << amount << ": " << coinChange(coins, amount) << std::endl;
}

```

```
    return 0;
}
```

3.3. Coin Change (Number of Ways)

Find the total number of distinct ways to make a given amount.

C++

```
#include <iostream>
#include <vector>

int coinChangeWays(const std::vector<int>& coins, int amount) {
    std::vector<int> dp(amount + 1, 0);
    dp[0] = 1; // Base case: one way to make amount 0 (with no coins)

    for (int coin : coins) {
        for (int j = coin; j <= amount; j++) {
            dp[j] += dp[j - coin];
        }
    }

    return dp[amount];
}

int main() {
    std::vector<int> coins = {1, 2, 5};
    int amount = 5;
    std::cout << "Ways to make " << amount << ": " << coinChangeWays(coins, amount) << "\n";
    return 0;
}
```

3.4. 0/1 Knapsack

Given items with weights and values, find the maximum value that can be carried in a knapsack of a given capacity.

DP (2D Array)

C++

```
#include <iostream>
#include <vector>
#include <algorithm> // For std::max

int knapsack(const std::vector<int>& wt, const std::vector<int>& val, int W) {
```

```

int n = wt.size();
std::vector<std::vector<int>> dp(n + 1, std::vector<int>(W + 1, 0));

for (int i = 1; i <= n; i++) {
    for (int w = 1; w <= W; w++) {
        if (wt[i - 1] <= w) {
            dp[i][w] = std::max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
        } else {
            dp[i][w] = dp[i - 1][w];
        }
    }
}
return dp[n][W];
}

int main() {
    std::vector<int> val = {60, 100, 120};
    std::vector<int> wt = {10, 20, 30};
    int W = 50;
    std::cout << "Max knapsack value: " << knapsack(wt, val, W) << std::endl;
    return 0;
}

```

Space Optimized (1D Array)

C++

```

#include <iostream>
#include <vector>
#include <algorithm> // For std::max

int knapsack_opt(const std::vector<int>& wt, const std::vector<int>& val, int W) {
    int n = wt.size();
    std::vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = std::max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }
    return dp[W];
}

int main() {
    std::vector<int> val = {60, 100, 120};
    std::vector<int> wt = {10, 20, 30};
    int W = 50;
}

```



```

std::cout << "Max knapsack value (optimized): " << knapsack_opt(wt, val, W)
return 0;
}

```

3.5. Longest Common Subsequence (LCS)

Find the length of the longest subsequence common to two strings.

C++

```

#include <iostream>
#include <string>
#include <vector>
#include <algorithm> // For std::max

int longestCommonSubsequence(const std::string& text1, const std::string& text2) {
    int m = text1.length();
    int n = text2.length();
    std::vector<std::vector<int>> dp(m + 1, std::vector<int>(n + 1, 0));

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1[i - 1] == text2[j - 1]) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}

int main() {
    std::string s1 = "abcde";
    std::string s2 = "ace";
    std::cout << "LCS length: " << longestCommonSubsequence(s1, s2) << std::endl;
    return 0;
}

```

3.6. Longest Increasing Subsequence (LIS)

Find the length of the longest subsequence of a given sequence such that all elements of the subsequence are sorted in increasing order.

C++

```

#include <iostream>
#include <vector>
#include <algorithm> // For std::max

int lengthOfLIS(const std::vector<int>& nums) {
    if (nums.empty()) return 0;
    int n = nums.size();
    std::vector<int> dp(n, 1);
    int max_len = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[i] > nums[j]) {
                dp[i] = std::max(dp[i], dp[j] + 1);
            }
        }
        max_len = std::max(max_len, dp[i]);
    }
    return max_len;
}

int main() {
    std::vector<int> nums = {10, 9, 2, 5, 3, 7, 101, 18};
    std::cout << "LIS length: " << lengthOfLIS(nums) << std::endl;
    return 0;
}

```

3.7. Edit Distance

Find the minimum number of operations (insert, delete, replace) required to convert one string to another.

C++

```

#include <iostream>
#include <string>
#include <vector>
#include <algorithm> // For std::min

int min3(int a, int b, int c) {
    return std::min({a, b, c});
}

int minDistance(const std::string& word1, const std::string& word2) {
    int m = word1.length();
    int n = word2.length();

```

```

std::vector<std::vector<int>> dp(m + 1, std::vector<int>(n + 1, 0));

for (int i = 0; i <= m; i++) dp[i][0] = i;
for (int j = 0; j <= n; j++) dp[0][j] = j;

for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (word1[i - 1] == word2[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1];
        } else {
            dp[i][j] = 1 + min3(dp[i - 1][j],          // Deletion
                                dp[i][j - 1],          // Insertion
                                dp[i - 1][j - 1]);      // Replacement
        }
    }
}
return dp[m][n];
}

int main() {
    std::string word1 = "horse";
    std::string word2 = "ros";
    std::cout << "Edit distance: " << minDistance(word1, word2) << std::endl;
    return 0;
}

```

3.8. Grid Paths with Obstacles

Similar to grid paths, but some cells are blocked.

C++

```

#include <iostream>
#include <vector>

int uniquePathsWithObstacles(std::vector<std::vector<int>>& obstacleGrid) {
    int m = obstacleGrid.size();
    int n = obstacleGrid[0].size();
    std::vector<std::vector<long long>> dp(m, std::vector<long long>(n, 0));

    if (obstacleGrid[0][0] == 1) return 0;
    dp[0][0] = 1;

    for (int i = 1; i < m; i++) {
        if (obstacleGrid[i][0] == 0) dp[i][0] = dp[i - 1][0];
    }
    for (int j = 1; j < n; j++) {

```

```

        if (obstacleGrid[0][j] == 0) dp[0][j] = dp[0][j - 1];
    }

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            if (obstacleGrid[i][j] == 0) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }
    }
    return dp[m - 1][n - 1];
}

int main() {
    std::vector<std::vector<int>> grid = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    std::cout << "Paths with obstacles: " << uniquePathsWithObstacles(grid) <<
    return 0;
}

```

3.9. Minimum Path Sum

Find the path from top-left to bottom-right in a grid with the minimum sum of values.

C++

```

#include <iostream>
#include <vector>
#include <algorithm> // For std::min

int minPathSum(std::vector<std::vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();

    for (int i = 1; i < m; i++) grid[i][0] += grid[i - 1][0];
    for (int j = 1; j < n; j++) grid[0][j] += grid[0][j - 1];

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            grid[i][j] += std::min(grid[i - 1][j], grid[i][j - 1]);
        }
    }
    return grid[m - 1][n - 1];
}

int main() {
    std::vector<std::vector<int>> grid = {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}};
    std::cout << "Minimum path sum: " << minPathSum(grid) << std::endl;
}

```

```
    return 0;
}
```

3.10. House Robber

You cannot rob two adjacent houses. Find the maximum amount you can rob.

C++

```
#include <iostream>
#include <vector>
#include <algorithm> // For std::max

int rob(const std::vector<int>& nums) {
    if (nums.empty()) return 0;
    int n = nums.size();
    if (n == 1) return nums[0];

    std::vector<int> dp(n);
    dp[0] = nums[0];
    dp[1] = std::max(nums[0], nums[1]);

    for (int i = 2; i < n; i++) {
        dp[i] = std::max(dp[i - 1], nums[i] + dp[i - 2]);
    }
    return dp[n - 1];
}

int main() {
    std::vector<int> houses = {2, 7, 9, 3, 1};
    std::cout << "Max amount to rob: " << rob(houses) << std::endl;
    return 0;
}
```

3.11. Maximum Subarray (Kadane's Algorithm)

Find the contiguous subarray with the largest sum.

C++

```
#include <iostream>
#include <vector>
#include <algorithm> // For std::max

int maxSubArray(const std::vector<int>& nums) {
    if (nums.empty()) return 0;
```

```

int max_so_far = nums[0];
int current_max = nums[0];

for (size_t i = 1; i < nums.size(); i++) {
    current_max = std::max(nums[i], current_max + nums[i]);
    max_so_far = std::max(max_so_far, current_max);
}
return max_so_far;
}

int main() {
    std::vector<int> arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    std::cout << "Maximum subarray sum: " << maxSubArray(arr) << std::endl;
    return 0;
}

```

3.12. Matrix Chain Multiplication

Find the most efficient way to multiply a chain of matrices.

C++

```

#include <iostream>
#include <vector>
#include <climits> // For INT_MAX

int matrixChainOrder(const std::vector<int>& p) {
    int n = p.size() - 1;
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(n + 1, 0));

    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (cost < dp[i][j]) {
                    dp[i][j] = cost;
                }
            }
        }
    }
    return dp[1][n];
}

int main() {
    std::vector<int> dims = {10, 30, 5, 60};
}

```

```

std::cout << "Min multiplications: " << matrixChainOrder(dims) << std::endl
return 0;
}

```

3.13. Longest Palindromic Subsequence

Find the length of the longest subsequence that is a palindrome.

C++

```

#include <iostream>
#include <string>
#include <vector>
#include <algorithm> // For std::max

int longestPalindromeSubseq(const std::string& s) {
    int n = s.length();
    std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));

    for (int i = n - 1; i >= 0; i--) {
        dp[i][i] = 1;
        for (int j = i + 1; j < n; j++) {
            if (s[i] == s[j]) {
                dp[i][j] = 2 + dp[i + 1][j - 1];
            } else {
                dp[i][j] = std::max(dp[i + 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[0][n - 1];
}

int main() {
    std::string str = "bbbab";
    std::cout << "Longest palindromic subsequence: " << longestPalindromeSubseq
    return 0;
}

```

3.14. Subset Sum

Determine if a subset of a given set of non-negative integers sums up to a given value.

C++

```

#include <iostream>
#include <vector>

```

```
bool isSubsetSum(const std::vector<int>& nums, int target) {
    std::vector<bool> dp(target + 1, false);
    dp[0] = true;

    for (int num : nums) {
        for (int j = target; j >= num; j--) {
            if (dp[j - num]) {
                dp[j] = true;
            }
        }
    }
    return dp[target];
}

int main() {
    std::vector<int> set = {3, 34, 4, 12, 5, 2};
    int sum = 9;
    if (isSubsetSum(set, sum)) {
        std::cout << "Found a subset with given sum" << std::endl;
    } else {
        std::cout << "No subset with given sum" << std::endl;
    }
    return 0;
}
```